勉強日記

日付

2026 年 04 月 14 日

学習内容

Amazon SageMaker Canvas と Model Registry の統合

- **概要:** SageMaker Canvas はコード不要（ノーコード）の機械学習サービスです。既存のモデルを Canvas にインポートして利用できますが、事前にそのモデルを SageMaker Model Registry に登録しておく必要があります。
- **試験ポイント:** Canvas へのカスタムモデルのインポート要件（直接ではなく、必ず Model Registry を経由する点）がよく問われます。ビジネスアナリスト向けのアーキテクチャ設問で選択肢に挙がりやすいです。
- **実務での使い方:** データサイエンティストが構築した精度の高いモデルを Model Registry でバージョン管理し、ビジネス部門のユーザーが Canvas の直感的な UI からそのモデルを呼び出してバッチ推論やシミュレーションを行うワークフロー（クロス機能連携）で活用されます。
- **副業での需要:** 需要度　。既存の分析モデルをビジネス部門で自律的に利用できるようにする社内 DX 支援や、ノーコードでの AI 活用推進（PoC・導入支援）の案件で、システム連携の知識が重宝されます。

RAG（検索拡張生成）におけるベクトル検索とプロンプト拡張

- **概要:** ユーザーの質問を Titan Embeddings 等でベクトル化してベクトル DB を検索し、最近傍の関連ドキュメントを特定します。特定後、ベクトル値ではなく「実際のテキストコンテンツ」を取得し、LLM へのプロンプトに組み込んで回答を生成させます。
- **試験ポイント:** RAG の仕組みにおいて「ベクトル DB から何を抽出して LLM に渡すか」が問われます。LLM が回答を生成できるよう、埋め込み（数値）ではなく「元のテキスト（コンテンツ）」をプロンプトとして渡す（拡張＝ Augment する）必要がある点が重要です。
- **実務での使い方:** Amazon Bedrock Knowledge Bases や Amazon OpenSearch Serverless 等を利用し、社内マニュアルなどを検索して根拠に基づいた回答を返す社内 QA ボットやカスタマーサポート向け生成 AI アプリケーションの構築で、標準的な構成となります。
- **副業での需要:** 需要度　。LLM を活用した RAG チャットボットの PoC や開発案件は非常に多く、ベクトル検索の概念や、LangChain 等によるプロンプトエンジニアリングのスキルと合わせて高単価案件の獲得に直結しやすい領域です。

SageMaker Model Monitor とデプロイ前後のデータ品質検査

- **概要:** SageMaker Model Monitor は、本番環境にデプロイされた「後」のモデル予測品質（データドリフトやバイアス等）を監視するサービスです。モデルをデプロ

イする「前」の、トレーニングデータの検査や学習時の誤分類への対処には使用できません。

- **試験ポイント:** 「デプロイ前の学習データの検証・誤分類への対応」を求めるユースケースにおいて、Model Monitor を解答とする誤りの選択肢（引っかけ）が頻出します。「デプロイ前（学習フェーズ）」か「デプロイ後（推論／運用フェーズ）」かの区別が回答の　になります。デプロイ前の評価には SageMaker Clarify などを選択するのが正解となります。
- **実務での使い方:** 実務の MLOps では、デプロイ前は SageMaker Data Wrangler や Clarify でデータ品質とバイアスをチェックし、デプロイ後はエンドポイントの入出力を Model Monitor で継続的に監視して、閾値を超えたら自動で再学習パイプライン（SageMaker Pipelines 等）をトリガーするルーチンを構築します。
- **副業での需要:** 需要度　。一度作ったモデルを腐らせずに本番環境で長期間安定して稼働させる「保守・運用（MLOps 基盤）の自動化」は運用案件として極めて需要が高く、継続的な収入源になりやすい重要なスキルです。

SageMaker with TensorBoard による学習プロセスのデバッグ・視覚化

- **概要:** SageMaker with TensorBoard は、モデルの「トレーニング中」の中間テンソル（アクティベーションや勾配など）を出力・視覚化して、モデル内部の状態を分析・デバッグするための機能です。
- **試験ポイント:** 「特定のクラスの適合率が低い原因のデバッグ」や「一部の画像が誤分類される理由のインサイト獲得」など、モデルのパフォーマンスのボトルネック（特にディープラーニングモデルの中間層の挙動）を解消・改善したい場合の最適なソリューションとして問われます。
- **実務での使い方:** PyTorch や TensorFlow などをバックエンドとする深層学習モデルの開発において、学習の収束が悪い場合や過学習が起きている際に、学習ジョブの指標をリアルタイムでモニタリングし、勾配や損失の推移からモデルアーキテクチャの調整やハイパーパラメータのチューニングを行う際に標準的に利用します。
- **副業での需要:** 需要度　。画像認識や自然言語処理などのディープラーニング精度改善・チューニング案件において重宝されます。単にコードが書けるだけでなく、学習プロセスの可視化ツールを用いて「なぜ精度が上がらないのか」を論理的に分析できるスキルは高単価案件の獲得に有益です。

SageMaker リアルタイム推論と EKS（自己構築環境）の比較

- **概要:** 推論基盤として EKS（Kubernetes）を自己構築すると運用のオーバーヘッドが大きくなり、メモリベースのスケーリングでは突発的なリクエスト増に迅速に対応しきれません。一方、SageMaker リアルタイムエンドポイントならフルマネージドで容易に高可用性と低レイテンシーを実現できます。
- **試験ポイント:** 「運用オーバーヘッドを最小限に抑える」「突発的なスパイクに迅速にスケーリングする」といった要件において、EKS や EC2 へのデプロイは誤りの選択肢（構成が複雑すぎる）となります。正解はマネージドな「SageMaker リアルタイム推論エンドポイント」を選択することです。
- **実務での使い方:** 企業全体で K8s (EKS) への環境統一という強いポリシーが無い限り、ML モデルの推論は SageMaker エンドポイントにホストし、前段のシステム（API Gateway + Lambda 等）から呼び出すアーキテクチャが運用保守の観点で

実務の最適解となります。

- **副業での需要:** 需要度　。限られた工数・予算で AI を組み込む PoC やスタートアップの案件では、複雑な EKS 運用を避けて SageMaker のマネージド機能をフル活用するインフラアーキテクチャ提案が非常に好まれます。

SageMaker Studio Classic と Notebook の使い分け / パイプラインの責務分解

- **概要:** SageMaker Studio Classic は実験管理からモデルデプロイパイプライン（SageMaker Pipelines 等）の構築まで ML ライフサイクル全体をカバーする統合 IDE です（単発の実験用の SageMaker Notebook とは異なります）。また、「データ処理・抽出（ETL）」は AWS Glue 等のインジェストパイプラインが担い、「承認から本番デプロイ」は SageMaker Pipelines が担うという責務の切り離しが重要です。
- **試験ポイント:** 用途を問う問題において、単なる分析には Notebook で足りますが、「CI/CD、バージョン管理、承認フローを含むパイプライン構築」が要件にある場合は Studio Classic (Pipelines) を選ぶのが正解です。また、データ準備タスクは Glue などの ETL 専用サービスで行い、モデル管理タスクと混同させないアーキテクチャ設計が頻出します。
- **実務での使い方:** データエンジニアが AWS Glue で構築したデータインジェスションパイプラインから特徴量を受け取り、MLOps エンジニアまたはデータサイエンティストが SageMaker Pipelines (Studio Classic 管理) を用いて再学習・自動デプロイパイプラインを回す、という役割分担とシステムの疎結合化がベストプラクティスとなります。
- **副業での需要:** 需要度　。Jupyter Notebook 上での単発のモデル構築スキルを持つ人材は多いですが、データ基盤から本番デプロイまでの完全な自動化（MLOps パイプライン化）をエンドツーエンドで設計・構築できる人材は不足しており、高単価かつ長期的な案件を獲得しやすくなります。

Redshift の動的データマスキングと匿名化アーキテクチャの比較

- **概要:** Redshift 上の機密データを元のデータ（ソース）を変更せずにクエリ実行時にリアルタイムで隠蔽するには「動的データマスキング」を使用します。Glue による事前 ETL、マテリアライズドビューによる変換、S3 へのアンロードと Athena の利用などは、データの複製や変換に伴う運用コスト・ストレージコストが発生するため要件を満たしません。
- **試験ポイント:** 「ソースデータを変更（複製）しない」「実装や運用作業を最小限に抑える」という要件下での PII マスキング手法が問われます。Redshift の動的データマスキングが正答となり、Glue でのジョブ作成やマテビューでの変換は「運用負荷が高い」「ソース非変更の要件違反」として誤りの選択肢（引っかけ）となります。
- **実務での使い方:** アナリストやデータサイエンティストが同じテーブルを参照する際、ロールベースのアクセス制御に合わせて動的マスキングポリシー（`CREATE MASKING POLICY`）を適用し、PII 列のみをマスクして提供することで、安全かつデータ複製を持たないセキュアな単一のデータソース（SSOT）運用を実現します。
- **副業での需要:** 需要度　。大規模データ分析環境の案件において、コンプライアンス（最小権限の原則）を遵守しつつ運用保守の手間を省くアーキテクチャの設計力は、高度なデータエンジニアリング案件（データマート構築や権限管理の最適化など）を獲

得する上での差別化要素になります。

Lake Formation によるデータアクセスの統合管理（S3 / Athena）

- **概要:** Athena 経由でのデータ分析において、S3 バケットポリシー単体で細粒度なアクセス制御（例: キャンペーンごとの制御）を行うとポリシーが複雑化し運用負荷が肥大化します。AWS Lake Formation を利用することで、S3 と Athena の両方に対する一元化された効率的なアクセス権限制御が可能になります。
- **試験ポイント:** 「運用オーバーヘッドを抑えつつ、Athena でのきめ細かい（テーブル、行、列レベルの）アクセス制御を行いたい」という要件に対し、S3 バケットポリシーや IAM ポリシーで個別に設定する選択肢は「運用負荷が高い（不正解）」として頻出します。Lake Formation による一元管理が正答となります。
- **実務での使い方:** 企業横断的なデータレイク基盤において、データセットへのアクセス権限を Lake Formation で一元管理します。データサイエンティストにはフルアクセスを付与し、ビジネス部門には特定の行・列（例：PII 以外）のみ許可するといったセキュアなガバナンスを容易に実現します。
- **副業での需要:** 需要度　。データ基盤構築（モダンデータスタック）の設計において、「構築後の安全な権限運用（データガバナンス）」までもカバーできるエンジニアは需要が非常に高く、Lake Formation の導入知識は高単価案件を獲得するための強い武器になります。

SageMaker 学習環境の最適化機能（起動時間・実行時間・コストの使い分け）

- **概要:** SageMaker での学習最適化は目的に応じて使い分けます。連続する反復学習の「起動オーバーヘッド（コンテナロード等）」を削減するには `Managed warm pools`。学習プロセス（実行フェーズ）自体の処理時間を短縮するには `Training Compiler`（コンパイル最適化）や `SMDDP`（分散データ並列）。コスト削減には中断対応（チェックポイント）が必須な `Managed Spot Training` を利用します。
- **試験ポイント:** 「連続するトレーニングジョブの『起動時間（インフラの待機時間）』を短縮したい」という要件に対し、`Managed warm pools` が正解となります。このシナリオにおいて、コストを抑える Spot や、実行自体の高速化を担う Compiler / SMDDP など目的が異なる機能が引っかけ選択肢として出題されます。
- **実務での使い方:** ハイパーパラメータ最適化（HPO）など反復的な試行時は Managed warm pools を有効にして作業効率を上げます。一方、大規模モデル本番学習時は SMDDP などで計算時間を短縮し、時間制約がない非同期のバッチ学習には Spot Training を適用する、といったトレードオフ（速度 vs コスト）に応じた環境設計を行います。
- **副業での需要:** 需要度　。機械学習基盤の構築案件において、プロジェクトの予算やスケジュール感に合わせて「速くする機能（warm pool, compiler）」「安くする機能（Spot）」を適切に組み合わせたインフラチューニングができると、開発者体験とコスト最適化の両面から顧客に高く評価されます。

target_precision ハイパーパラメータと偽陽性の最小化（Precision の制御）

- **概要:** `target_precision` ハイパーパラメータは、モデルが目標とする適合率（Precision）を制御します。「正と予測したもののうち、実際に正である割合」を高め

るように学習を促すため、この値を大きくすると結果的に「偽陽性（False Positive: 誤検知）」を最小化できます。

- **試験ポイント:** 「誤検知（偽陽性）を最小限に抑えたい」というビジネス要件（例：正常なメールをスパムとして弾いてしまうのを防ぐ等）が出題された場合、`target_precision` の値を増やすアプローチが正解となります。再現率（Recall）とのトレードオフの文脈で頻出します。
- **実務での使い方:** 医療のアラートシステムや不正決済検知など、「誤ってアラートを出すこと（偽陽性）」への運用・ビジネス上のコストが非常に高い業務領域において、実環境の要件に合わせて `target_precision` の閾値を高めに設定し、確実に該当するものだけを検知するようチューニングします。
- **副業での需要:** 需要度 。単にモデルの精度（Accuracy）を上げるだけでなく、クライアントのビジネス上の「誤検知のコスト」と「見逃しのコスト」を天秤にかけ、Precision と Recall のトレードオフを理解した上でモデルの評価・調整を提案できるスキルは、PoC やデータ分析案件で高く評価されます。

SageMaker Model Registry Collections とモデルの階層的管理・発見可能性

- **概要:** 大量の ML モデルを管理・検索（発見可能性の向上）しやすくするため、「モデルグループ（関連するモデルバージョンの集まり）」をさらに上位の概念で論理的に束ねる機能が `Model Registry Collections` です。既存のモデルグループやアーティファクトへの参照を壊すことなく、階層的な整理（カテゴリ分け等）が可能です。
- **試験ポイント:** 「既存のアーティファクト構造に影響を与えずに、スケールする形で大量のモデルを整理・分類し、発見可能性を向上させたい」という要件では `Model Registry Collections` が正答となります。カスタムタグによる管理や、プロセスの依存関係を追跡するだけの `ML Lineage Tracking` は最適解ではない引っかけ選択肢として出題されます。
- **実務での使い方:** 企業内で複数の AI プロジェクトが並行して走る環境下において、「部門別」や「AI タスク別（NLP、画像認識など）」といった上位カテゴリとして Collections を作成し、各プロジェクトのモデルグループをそこに紐づけます。これにより、データサイエンティストが社内の既存モデルを効率的に検索・再利用（ModelOps）できるようになります。
- **副業での需要:** 需要度 。MLOps 基盤の構築において、単一モデルの管理から一歩進んで「全社的なモデル管理ポータル（横断的なモデルカタログ）」を設計するような、エンタープライズ向けのアーキテクチャ設計案件で提案できる知識として重宝されます。

Amazon ECR と EKS を活用したコンテナベース・アーキテクチャの特徴

- **概要:** ECR はイメージ暗号化・スキャン機能を備えたフルマネージドな Docker コンテナレジストリで、EKS はコントロールプレーンの管理を AWS にオフロードできるマネージド Kubernetes サービスです。両者はネイティブに統合されており、IAM による強固なアクセス制御のもとシームレスなイメージ連携が可能です。
- **試験ポイント:** 「オンプレミスの Kubernetes 環境や既存の Docker イメージを、最小限の変更で AWS に移行したい」という要件における最適なアーキテクチャとして問われます。既存のマニフェストファイルをそのまま利用でき、運用オーバーヘッドを削減できる点がポイントになります。

- **実務での使い方:** 機械学習モデルの推論用コンテナや前処理のマイクロサービスをデプロイする際、ECR にコンテナイメージを格納し、IAM ロールベースで権限を分離した EKS クラスターから安全に Pull して実行する、モダンなクラウドネイティブ推論基盤として構成されます。
- **副業での需要:** 需要度 。K8s（Kubernetes）エコシステムやコンテナ技術自体は DevOps 案件の主戦場ですが、機械学習エンジニアが ECR/EKS を使いこなせることで、「インフラからモデルの本番運用（MLOps）まで一貫して任せられる」と高く評価され、案件の獲得ハードルを下げる重要な要素になります。

AWS Glue DataBrew とクラス不均衡対応（オーバーサンプリング）の限界

- **概要:** AWS Glue DataBrew は視覚的な UI でデータクリーニングや特徴量エンジニアリングを行う ETL サービスですが、SMOTE のような機械学習特有の「クラス不均衡を解決するオーバーサンプリング機能」は提供していません。
- **試験ポイント:** 「データセットのクラス不均衡（不均衡データ）を解消したい」という要件において、DataBrew によるオーバーサンプリングを解決策とする選択肢は「該当機能が存在しないため不正解（引っかけ）」となります。不均衡データの解消には SageMaker Data Wrangler やカスタムスクリプトを利用するのが正解です。
- **実務での使い方:** 欠損値補完や日付フォーマットの変換、異常値の除去といった基礎的なデータ準備タスクは DataBrew に任せ、そのあとのモデル学習に直結する前処理（オーバーサンプリングやアンダーサンプリング）は SageMaker 上の Python スクリプト処理等に切り分けることで、最適なパイプラインを構築します。
- **副業での需要:** 需要度 。DataBrew 単体の深い知識が単独で高単価案件につながることは稀ですが、ツールの「できること」と「できないこと（限界）」を正しく把握し、SageMaker エコシステムと組み合わせて破綻のないデータ処理パイプラインを設計できる能力は、実プロジェクトを円滑に進める上で不可欠です。

Amazon EMR クラスターのノードタイプ解説とスポットインスタンス戦略

- **概要:** EMR クラスターは「プライマリノード（管理・スケジューリング）」「コアノード（データ保持＋演算）」「タスクノード（演算専用・データ非保持）」の 3 種で構成されます。データ損失を防ぎつつコストを抑えるには、データを持たない「タスクノード」に限定して Spot インスタンスを利用します。
- **試験ポイント:** 「データ損失リスクを許容せずに EMR のコスト最適化を行いたい」という要件で頻出します。「プライマリとコアには On-Demand インスタンスを使用し、タスクノードにのみ Spot インスタンスを使用する」が正解です。コアノードに Spot を使用する選択肢は、中断時にデータが失われるリスクがあるため不正解（引っかけ）となります。
- **実務での使い方:** 大規模な分散データ処理（Apache Spark など）を行う際、ベースとなるリソースとしてプライマリと最低限のコアノードを確実に On-Demand で確保し、ジョブのピーク時にスケーリングする演算力（タスクノード）として Spot を割り当てるアーキテクチャが、コストと安定性を両立する定石です。
- **副業での需要:** 需要度 。大規模データエンジニアリング案件（ペタバイト級のデータレイク基盤や ETL 処理）において、EMR を適切にサイジングし、大幅なコスト削減（最大 90% の Spot 活用など）を提案・実装できるエンジニアは、顧客のインフラ維持費を直接削減できるため重宝されます。

SageMaker Data Wrangler と AWS Glue データカタログの連携（カタログ接続）

- **概要:** SageMaker Data Wrangler のデータインポートには、取得時点のスナップショットを読み込む「ダイレクト接続」と、メタデータを一元管理する「カタログ接続」があります。カタログ接続を利用すると、AWS Glue データカタログ（クローラーによって定期的にスキーマ変更が自動検知・更新されるメタデータリポジトリ）の最新情報を参照して、S3 や Redshift 等のデータを統合的に扱うことができます（最大 1,000 列までインポート可能）。
- **試験ポイント:** 「データソースの継続的な更新やスキーマ変更を自動で検知して最新の状態を維持しつつデータをインポートしたい」という要件において、スナップショットであるダイレクト接続は不正解となり、Glue クローラーと連携した「カタログ接続」が正答となります。また他のサービス（Athena, Redshift Spectrum, EMR 等）とメタデータを共有する文脈でも非常によく出題されます。
- **実務での使い方:** 企業横断のデータレイク環境において、S3 上の生データのスキーマを Glue クローラーで自動検知・カタログ化しておきます。データサイエンティストは Data Wrangler 上で SQL を用いて直感的にカタログ（Database・Table）を指定し、エンジニアと同じ一貫したメタデータ構造を利用して特徴量エンジニアリング（前処理）を実行します。
- **副業での需要:** 需要度　。機械学習の前処理にとどまらず、データレイク設計全体における「Glue データカタログ」の重要性を理解し、分析官（Data Wrangler / Athena ユーザー）向けに一貫性のあるメタデータ基盤・参照アーキテクチャを提案できるスキルは、データ基盤構築案件で手堅い評価を得られます。

Lambda と SageMaker Clarify を活用したサーバーレスなバイアスドリフト監視

- **概要:** 機械学習モデルのバイアスドリフト（偏り）を定期的に監視する際、EventBridge でスケジュールを設定し、AWS Lambda を起動して SageMaker Clarify ジョブを呼び出す（オンデマンド実行する）サーバーレスアーキテクチャを組むことで、極めてスケーラブルかつ運用オーバーヘッドが最小限の監視ワークフローが実現します。
- **試験ポイント:** 「運用オーバーヘッド（管理の手間）を最小限に抑えつつ、自動でバイアス監視のワークフローをスケジュール・オンデマンド実行したい」という要件において、「EventBridge + Lambda + SageMaker Clarify」の組み合わせが最適解（正答）になります。EC2 等のインスタンス上で cron を動かすような選択肢は運用負荷が高いため不正解です。
- **実務での使い方:** 推論ログが蓄積される実践的な本番環境において、毎週あるいはデータ量が一定に達したイベントをトリガーとして EventBridge → Lambda を発火させ、Clarify によるバイアス検証ジョブを自動実行・レポート生成させる完全自動化パイプライン（MLOps の一環）として広く採用されます。
- **副業での需要:** 需要度　。サーバーレス（EventBridge + Lambda）をトリガーにして重い ML 処理（SageMaker 等）をキックする設計パターンの習得は、機械学習エンジニアリングのみならずクラウドアーキテクチャ全般の定石です。インフラと MLOps の両軸を理解して実装できるエンジニアは市場価値が高く、継続的な運用保守案件を獲得しやすくなります。

SageMaker の推論オプションの使い分け（サーバーレス、リアルタイム、非同期、バッチ）

- **概要:** SageMaker の推論構成は要件に応じて使い分けます。トラフィックが不規則でアイドル時コストをゼロ（スケールイン・ゼロ）にしたい場合は サーバーレス推論（コールドスタート対策で Provisioned Concurrency 指定可）。極めて低いレイテンシで常時応答が必要なら リアルタイム推論（最小 1 台常時稼働）。最大 1GB 等の巨大ペイロードや長時間処理ならキューイングする 非同期推論（これもゼロスケール可能だが Web の即時応答には不向き）。S3 上のオフライン一括処理なら バッチ変換 となります。
- **試験ポイント:** 「トラフィックが突発的（スパイク）で予測困難であり、利用しない時間帯のインフラコストをゼロにしたい」というシナリオでは サーバーレス推論 が正解として出題されます。非同期推論もゼロにスケールダウンできますが、「即時のリアルタイム応答が必要」という要件がつくと不正解（引っかけ）になります。
- **実務での使い方:** アクセスが集中する時間帯が限定的な BtoB の社内向けツールや PoC アプリでは、圧倒的なコスト削減効果を生むサーバーレス推論によるアーキテクチャが重宝されます。逆に、EC サイトの商品レコメンドや広告配信など、数ミリ秒単位の継続的な応答速度がビジネスに直結するインフラでは、リアルタイム推論（＋オートスケーリング）を構築するのが必須の定石です。
- **副業での需要:** 需要度　。MLOps 基盤の提案案件において、ビジネス側の要件（許容されるレイテンシと予算の兼ね合い）をヒアリングし、AWS での最適な推論オプション（特にサーバーレスによる運用費の劇的な削減）を適材適所で設計・提案できるエンジニアリングスキルは、アーキテクトとして非常に高く評価されます。

SageMaker AI モデル構築オプションの使い分け（JumpStart、スクリプトモード等）

- **概要:** SageMaker AI には、導入ハードルの低い順に `JumpStart`（学習済みモデル）、組み込みアルゴリズム、スクリプトモード（フレームワークコンテナ）、カスタム `Docker` イメージ の 4 段階の構築オプションがあります。用途や移行コスト、カスタマイズ性に応じて選択します。
- **試験ポイント:**「既存の Python スクリプトを最小限の変更で持ち込みたいが、Docker イメージの管理負荷は避けたい」という要件に対し、スクリプトモード が最適解となります。対照的に、`Canvas` はノーコード専用でスクリプト持ち込みができない点、`JumpStart` はモデルハブとしての役割が強い点が問われます。
- **実務での使い方:** 素早い検証には JumpStart を使い、独自のモデルロジックが必要な本格開発ではスクリプトモードを採用します。スクリプトモードなら `requirements.txt` で依存ライブラリを管理でき、インフラのパッチ当てや管理を AWS に任せつつ、SageMaker 外で作成したコードとほぼ同じ形式で実行可能です。
- **副業での需要:** 需要度　。既存のローカル学習コードをクラウド上のスケーラブルな学習システムへ迅速に移行・最適化できるスキルは、スピード感が求められる副業案件で非常に重宝されます。特にカスタムイメージ作成の手間を省くスクリプトモードの提案は、工数削減の観点から高く評価されます。

モデルの性能低下要因（コンセプトドリフト・データドリフト・特徴量ドリフト）

- **概要:** 運用中のモデル性能（F1 スコアなど）が低下する主な原因分類です。入力データ全体の分布変化である「データドリフト」、特定特徴量の分布変化である「特徴量ド

リフト」、データと目的変数（正解ラベル）の根本的な関係性が変わる「コンセプトドリフト（概念ドリフト）」があります。

- **試験ポイント:** 「稼働開始から数ヶ月後に F1 スコア等の評価指標が低下した。考えられる主な原因は何か」というトラブルシューティングの問題で頻出します。市場変化やユーザーの行動変化などにより、推論データの傾向が変わったことによる「コンセプトドリフト」や「データドリフト」が正答として選ばれます。
- **実務での使い方:** SageMaker Model Monitor（Data Quality Monitor や Model Quality Monitor）を利用し、学習時のベースラインデータと推論時のデータ間の統計的距離を定期的に計測します。ドリフトのアラートが発砲された場合、最新のデータを用いて再学習（Continuous Training）を行い、モデルを適宜切り替えて性能低下を防ぐ仕組みを構築します。
- **副業での需要:** 需要度　。AI 開発案件において「モデルを作って終わり」にせず、将来必ず起こる劣化要因を定義し、監視・再学習サイクルという長期的な ML ライフサイクル管理（MLOps）まで提案できるエンジニアは、単なる実装者以上のアーキテクトとして顧客・クライアントからの信頼を強固なものにできます。

SageMaker ノートブックインスタンスの権限一元管理（単一 IAM ロールの活用）

- **概要:** チームで複数の SageMaker ノートブックインスタンスを利用する際、インスタンスごとに個別の権限を管理すると運用オーバーヘッドが肥大化します。これを防ぐため、チーム共通の「単一の IAM ロール」を作成して各ノートブックインスタンスにアタッチし、権限の変更・追加をその 1 つのロールのみで行う方法がベストプラクティスです。
- **試験ポイント:** 「運用オーバーヘッドを抑えつつ、複数のノートブックへの新しい権限追加を一元化・効率化したい」という要件で頻出します。正解は「単一の IAM ロールを作成し全インスタンスにアタッチする」になります。引っかけとして「IAM グループにアクセス権を付与する」や「IAM ユーザーをアタッチする」といった選択肢が出ますが、これらはリソースの実行権限（インスタンスプロファイル）としては機能せず用途が異なるため不正解です。
- **実務での使い方:** 企業でデータ分析チームを組成する際、例えば「分析用 S3 バケットへの Read/Write 権限」や「特定の Glue データベースへのアクセス権」を持った単一の『DataScientist-Notebook-Role』を作成し、チームメンバー全員のインスタンス起動用デフォルトロールとして設定・一元運用します。
- **副業での需要:** 需要度　。ML エンジニアリング案件では「モデルが作れる」こと以上に、「セキュアで運用負荷の低い開発環境（AWS IAM の適切な設計）をチーム全体に提供できるか」が問われます。IAM ロールとユーザー・グループの適切な使い分けは、ML 基盤の設計・構築案件において必須となるインフラ知識です。

SageMaker Clarify による特徴量アトリビューションドリフトの継続的監視

- **概要:** SageMaker Clarify（Model Monitor との連携）は、本番エンドポイントにおける「特徴量アトリビューションドリフト（予測に寄与する特徴量の重要度の変化）」を継続監視できます。具体的には、SHAP 値による特徴量重要度ランキングの学習時と推論時の差を NDCG (正規化割引累積利得) スコアで定量化します。
- **試験ポイント:** 「長期的な自動化ソリューションとして、再トレーニングのタイミングを自動で検知したい」という要件で頻出します。Clarify をスケジュール実行して監

視し、NDCG スコア（例：0.90 未満）の低下をトリガーに CloudWatch アラートを発報させるアーキテクチャが正答となります。手動での定期的な分析（Jupyter Notebook での都度確認など）は「自動化ソリューション」の要件を満たさず不正解です。

- **実務での使い方:** トレンド変化が早いレコメンドモデルや金融の与信モデルにおいて、ベースライン（学習データ）から特徴量の「効き方（SHAP 値の順位）」が変化していないかを日次/週次でモニタリングします。スコア悪化時に CloudWatch Alarm を発火させ、EventBridge 経由で SageMaker Pipelines を起動するといった完全自動の再学習（Continuous Training）サイクルを構築します。
- **副業での需要:** 需要度　。精度指標（正解率や F1 スコア等。正解ラベルが後から得られないと計算不可）の集計を待つことなく、「モデルの判断根拠（特徴量の重要度）が変わった」時点ですぐに劣化の兆候を捉え自動リカバリできる高度な MLOps の設計は、大規模データ基盤や AI プロダクトのアーキテクトとして非常に高く評価されます。

重複レコードの統合（名寄せ）における AWS Glue FindMatches の活用

- **概要:** 異なるフォーマットや表記揺れが含まれる大規模なデータセットから重複レコードを自動で検出・名寄せするための、機械学習ベースの専用変換機能が `AWS Glue FindMatches` です。Data Wrangler も ETL ツールですが、複数属性にまたがる高度な重複検出には独自コードの開発が必要となるため向きません。
- **試験ポイント:** 「コード開発を最小限に抑止し、視覚的なインターフェースで大規模なデータセット内の類似・重複レコード（名寄せ処理）を検出したい」という要件において `AWS Glue FindMatches` が正答となります。Data Wrangler は「重複検出に特化した組み込み機能がない（カスタムロジックが必要）」という理由で不正解の選択肢になります。
- **実務での使い方:** 複数の別システムから集約された顧客マスターデータ（例：「株式会社A」と「A 社」などの表記揺れ）を統合するデータレイク構築において、AWS Glue の ETL ジョブ内に FindMatches の変換ステップを組み込みます。カスタムの自然言語処理アルゴリズムを手組みすることなく、高精度なデータクレンジングをスケールさせることが可能です。
- **副業での需要:** 需要度　。データ基盤開発や CDP（カスタマーデータプラットフォーム）構築案件において、「名寄せ処理」は最も泥臭く工数がかかる課題です。ここに高価な外部 SaaS や複雑な手製スクリプトを入れるのではなく、AWS ネイティブな FindMatches で開発工数と運用負荷を抑えるアーキテクチャ提案ができると顧客から非常に喜ばれます。